Programming 2

Tutorial 9

Exercise 1 (required)

File `CountLetters.java` contains a program that reads a word from the user and prints the number of occurrences of each letter in the word. Save it to your package and study it, then compile and run it to see how it works. While reading the code, note that the word is converted to all upper case first, then each letter is translated to a number in **the range** `0..25` (by subtracting `'A'`) for use as an index. No test is done to ensure that the characters are in fact letters.

Step 1. Run `CountLetters` and enter a phrase, that is, more than one word with spaces or other punctuation in between. It should throw an `ArrayIndexOutOfBoundsException`, because a non-letter character will generate an index that is not between `0` and `25`. It might be desirable to allow non-letter characters, but not count them. Of course, you could explicitly test the value of the character to see if it is between `'A'` and `'Z'`. However, an alternative is to go ahead and use the translated character as an index, and catch an `ArrayIndexOutOfBoundsException` if it occurs. Since you don't want to do anything when a non-letter occurs, the handler will be empty. Modify this method to do this as follows:

- Put the body of the first `for` loop in a try.
- Add a `catch` block that catches the exception, but don't do anything with it.
- Compile and run your program.

Step 2. Now modify the body of the `catch` block so that it prints a useful message (e.g., "Not a letter") followed by the exception. Compile and run the program. Although it's useful to print the exception for debugging, when you're trying to smoothly handle a condition that you don't consider erroneous, you often don't want to. In your `print` statement, replace the exception with the character that created the out-of-bounds index. Run the program again; much nicer!

Exercise 2 (required)

File `ParseInts.java` contains a program that does the following:

1. Prompts for and reads in a line of input
2. Uses a second `Scanner` to take the input line one token at a time and parses an integer from each token as it is extracted.
3. Sums the integers.
4. Prints the sum.

- Save `ParseInts` to your directory and compile and run it. If you give it the input

```
10 20 30 40
```

it should print

```
The sum of the integers on the line is 100.
```

- Try some other inputs as well. Now try a line that contains both integers and other values, e.g.,

```
We have 2 dogs and 1 cat.
```

- You should get a `NumberFormatException` when it tries to call `Integer.parseInt` on `"We"`, which is not an integer. One way around this is to put the loop that reads inside a `try` and catch the `NumberFormatException` but not do anything with it. This way if it's not an integer it doesn't cause an error; it goes to the exception handler, which does nothing. Do this as follows:
  - ✓ Modify the program to add a try statement that encompasses the entire `while` loop. The `try` and opening `{` should go before the `while`, and the `catch` after the loop body. Catch a `NumberFormatException` and have an empty body for the `catch`.
  - ✓ Compile and run the program and enter a line with mixed integers and other values. You should find that it stops summing at the first non-integer, so the line above will produce a sum of `0`, and the line `"1 fish 2 fish"` will produce a sum of `1`. This is because the entire loop is inside the `try`, so when an exception is thrown the loop is terminated. To make it continue, move the `try` and `catch` inside the loop. Now when an exception is thrown, the next statement is the next iteration of the loop, so the entire line is processed. The *dogs-and-cats* input should now give a sum of `3`, as should the *fish input*.

Exercise 3 (required)

File `Factorials.java` contains a program utilizing the `factorial` method from the `MathUtils` class to compute factorials of user-input integers. Save the files to your package, study the code, compile, and run `Factorials` to observe its functionality. Test with various positive integers, then try a negative number. You'll find it works for small positive integers (smaller than 17), but returns a large negative value for larger integers and `1` for negative integers.

1. Correcting the behavior for negative integers:
   - Modify the `factorial` method header to declare that it can throw an `IllegalArgumentException`.
   - Adjust the `factorial` method body to throw an `IllegalArgumentException` if the argument is negative. Pass a specific message to the constructor indicating the issue.
   - Compile and run `Factorials` to observe the program throwing an exception for negative numbers.
2. Correcting the behavior for values over `16`:
   - Enhance the `factorial` method to also check for arguments over `16`, signaling an `IllegalArgumentException`. Provide distinct messages for both negative and large arguments to clarify the issue.